NS-3 NB-IoT Simulator User Guide

Introduction

This guide provides students with a fully prepared environment for running NS-3 NB-IoT simulations using Docker. The goal is to ensure a consistent, reproducible setup across all lab machines and student laptops.

The NS-3 simulator is packaged inside a Docker container, eliminating the need for manual installation of dependencies or compilation on your local system.

1. Overview

The NS-3 NB-IoT simulator is provided as a Docker-based environment. Docker ensures that every student runs the same version of NS-3 with identical dependencies.

You will:

- Install Docker
- Build the NS-3 simulator container
- Run the container
- Execute NB-IoT simulations

2. Installing Docker

Docker allows applications to run inside isolated containers. Follow the official Docker installation guide:

https://docs.docker.com/engine/install/

If you prefer you can install its GUI (https://docs.docker.com/desktop/). In either case, choose your operating system (Windows, macOS, Linux) and follow the steps provided.

Verify Installation

Open a terminal and run:

```
docker --version
docker compose version
```

You should see version numbers printed.

3. Project Structure

Your instructor will provide a folder containing:

- .vscode
- docs
- ns3-nbiot
- docker-compose.yaml
- Dockerfile

These files define the NS-3 simulator environment.

You do not need to modify them.

4. Building the NS-3 Simulator Container

Open a terminal and navigate to the project folder:

```
cd <project-folder>
```

Build the container:

```
docker compose build
```

This step:

- Downloads Ubuntu 22.04
- Installs NS-3 dependencies
- Clones the NB-IoT NS-3 repository
- Builds NS-3 using waf

The *build* may take several minutes and your computer needs to be connected to the Internet.

5. Running the Container

Start an interactive session inside the NS-3 environment:

```
docker compose run ns3
```

You will be placed inside the workdir defined in docker-compose.yaml.

By default, it is **/opt/ns3-nbiot**.

This directory contains the full NS-3 source tree.

There is another option to run the container and automatically remove it from the disk when you exit it. Use it with care, because the container is destroyed with no leftovers (you lose anything you left inside the container's disk), only the original image is intact.

```
docker compose run --rm ns3
```

6. Running NS-3 Simulations

Inside the container, simulations are executed using waf:

```
./waf --run <program-name>
```

For example, to run a scratch simulation:

```
./waf --run scratch/my-simulation
```

7. Exiting and Re-entering the Container

To exit the NS-3 environment:

```
exit
```

To re-enter later:

```
docker compose run ns3
```

8. Optional Cleanup

Remove the built image:

```
docker image rm ns3-simulator-ns3
```

Remove stopped containers.

```
docker container prune
```

If you run the container with the --rm option, every time you “exit” the container, it is automatically removed (and you cannot reenter it).

9. Final Example: Running an example

In this lab, you will explore energy consumption and state transitions in Cellular IoT (5G mMTC) devices using the ns-3 network simulator. You will run a simulation scenario where an IoT device equipped with an energy harvester sends small data packets over a cellular network to a remote server.

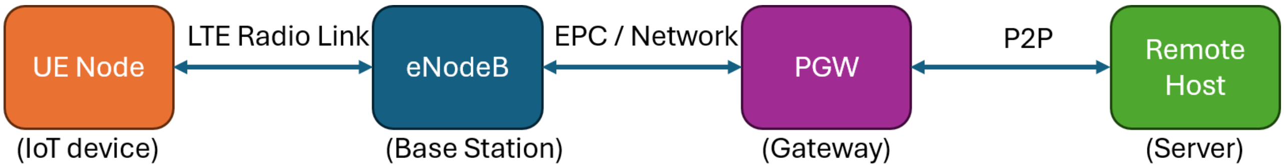
Lab Objectives

By the end of this lab, you will be able to:

- Understand the architecture of an LTE/5G Cellular IoT simulation in ns-3.
- Run customized network simulations inside a Docker container.
- Observe state changes (ACTIVE vs. INACTIVE) driven by the energy model.
- Locate and inspect generated log files and packet traces.

Network Scenario Architecture

The simulation (nb-scenario3.cc) models a typical IoT deployment:



1. **User Equipment (UE):** A static IoT device placed in a cell with a radius of up to 2500m. It operates on an **Energy Markov** battery model starting at 3.3 V.
2. **eNodeB (Base Station):** Fixed at the center of the cell to handle cellular coverage.
3. **Core Network & Remote Host:** Packets are routed through the LTE Evolved Packet Core (EPC) to a remote host over a high-speed link (100 Gbps).
4. **Traffic Profile:** The IoT device sends 49-byte UDP packets (simulating a 32-byte mMTC payload + CoAP/DTLS headers) every 1 minute.

Ensure your Docker container environment is running. To prevent long simulation times during class, limit the simulation time to **60 seconds** using the --simTime parameter. By default, this parameter is set to 180 minutes. Execute the following command inside your container shell:

```
cd /opt /ns3-nbiot
./waf --run "scratch/nb-scenario3.cc --simTime=60"
```

We can define the number of Ues (i.e., IoT devices in the simulation) using the parameter –num_ues. This parameter is, by default, equal to 1.

Inspecting Simulation Outputs & Logs

Once the simulation finishes, output files and traces are automatically organized into structured log directories. Inside the container, all log files are written to “logs” folder inside the ns3 directory.

This container folder is mapped to ./logs on your local host system so you can easily view files using your local text editor or tools.

The folder path automatically structures outputs based on your run configuration:

```
logs / markov / 1_60000000000_0_0 / 22_07_2026_07_05_08
    0_1_DataRecep.log
    0_1_DataTrans.log
    0_1_Energy.log
    0_1_lena-simple-epc-0-2.pcap
    0_1_lena-simple-epc-0-3.pcap
    0_1_lena-simple-epc-1-1.pcap
    0_1_lena-simple-epc-1-2.pcap
    0_1_lena-simple-epc-1-3.pcap
    0_1_lena-simple-epc-2-1.pcap
    0_1_lena-simple-epc-3-1.pcap
    0_1_lena-simple-epc-4-2.pcap
    0_1_MAC.log
    0_1_RRC.log
    0_1_Spectral_Downlink.log
    0_1_Spectral_Uplink.log
    0_1_ueMAC.log
```

Log & Trace Output

1. **Console Output:** You will see state transition events printed directly to standard output as the energy levels shift:

```
Waf: Entering directory `/opt/ns3-nbiot/build'
Waf: Leaving directory `/opt/ns3-nbiot/build'
Build commands will be stored in build/compile_commands.json
'build' finished successfully (4.652s)
State ACTIVE  : 1
State INACTIVE: 0
Remote host address: 1.0.0.2
PGW address:
 * Interface #0: 127.0.0.1
 * Interface #1: 7.0.0.1
 * Interface #2: 14.0.0.5
 * Interface #3: 1.0.0.1
Node#0 Position:1394.71,573.577,1.5
EnergyMarkov(0): State machine initialized to 1 with delta0 = 0.7, delta1 = 0.1 at 0 s
EnergyMarkov(0): State machine initialized to 1 with delta0 = 0.7, delta1 = 0.1 at 0 s
EnergyMarkov(0): State machine initialized to 1 with delta0 = 0.7, delta1 = 0.1 at 0 s
Started computation at Wed Jul 22 07:25:48 2026
Number of UEs: 1
EnergyMarkov(5): Updating remaining energy at 0 s
EnergyMarkov(5): State remained as 1 at 0 s
EnergyMarkov(5): Updating remaining energy at 0.01 s
EnergyMarkov(5): State remained as 1 at 0.01 s
EnergyMarkov(5): Updating remaining energy at 0.02 s
EnergyMarkov(5): State remained as 1 at 0.02 s
EnergyMarkov(5): Updating remaining energy at 0.02 s
EnergyMarkov(5): State remained as 1 at 0.02 s
EnergyMarkov(5): Updating remaining energy at 0.02 s
EnergyMarkov(5): State remained as 1 at 0.02 s
EnergyMarkov(5): Updating remaining energy at 0.03 s
```

2. **PCAP Traces:** File captures (such as lena-simple-epc-*.pcap) are stored in the log directory. You can open these with **Wireshark** on your host machine under ./logs/ to analyze packet flows.

Practice

1. Run the simulation for 30 minutes with 2 IoTs with default settings. Note how many state transitions occur in the console output.

2. Check the log folder under "./logs" on your local machine to verify that your new run generated a timestamped folder containing PCAP files.

3. Open the pcap file using Wireshark. Can you identify the transmitted packets?

4. Open the "Energy.log". Plot energy vs. Time. Analyse your findings.

Conclusion

You now have a fully functional NS-3 NB-IoT simulation environment using Docker. This setup ensures consistency across all student machines and simplifies the process of running simulations during lab sessions.